

## Table des matières

|                                                              |   |
|--------------------------------------------------------------|---|
| Cours 02 — Les formats de fichiers et le lac bronze.....     | 1 |
| Objectifs.....                                               | 1 |
| 1. Ligne contre colonne.....                                 | 2 |
| 2. Cinq formats, ce qu'ils coûtent et ce qu'ils rendent..... | 2 |
| 3. Parquet, de près.....                                     | 3 |
| 4. La règle du bronze.....                                   | 4 |
| 5. Le stockage objet et le partitionnement.....              | 5 |
| 6. DuckDB sur le lac.....                                    | 6 |
| Points clés à retenir.....                                   | 7 |
| Exercices.....                                               | 8 |

## Cours 02 — Les formats de fichiers et le lac bronze

*Théorie · Module 2/12 — Formation Data Engineering · ~25 min · débutant*

Où vous êtes : source → ingestion → **lac** → entrepôt → restitution. Le Cours 02 s'occupe du lac : la forme des fichiers qu'on y range, et la façon de les ranger.

En 30 secondes :

- Un fichier en lignes (CSV, JSON) se lit en entier ; un fichier en colonnes (Parquet) ne lit que les colonnes demandées, et saute ce que ses statistiques excluent.
- Un CSV ou un JSON ne transporte aucun type : le bronze déclare ses types au lieu de les laisser deviner, et garde en texte ce qu'une conversion abîmerait.
- Dans un lac, le chemin d'un fichier sert d'index : partitionner par jour livré permet de ne lire qu'un jour, et de le remplacer seul. Trop de petits fichiers coûtent cher.

L'idée du cours. Le Cours 01 a regardé les sources telles quelles, et constaté ce que coûte un type deviné. Ici, on range pour de bon. Le lac garde l'historien et la télémétrie en Parquet, avec des types déclarés, un dossier par jour livré. Chaque choix se mesure sur Boréal, en octets et en secondes, et chaque piège se montre sur une vraie colonne.

## Objectifs

- Expliquer ce qui distingue un format en lignes d'un format en colonnes, et quand chacun convient.
- Comparer CSV, JSON lines, Avro, Parquet et ORC sur ce qu'ils coûtent et ce qu'ils rendent.
- Décrire l'intérieur d'un fichier Parquet : groupes de lignes, blocs de colonnes, statistiques, encodage et compression.
- Appliquer la règle du bronze : déclarer les types, et garder en texte ce qu'une conversion abîmerait.
- Partitionner un lac par chemin, choisir la clé de partition, et reconnaître le piège des petits fichiers.
- Écrire et interroger du Parquet avec DuckDB, et lire ce qu'un fichier dit de lui-même.

## 1. Ligne contre colonne

Une table peut s'écrire de deux façons sur un disque. En lignes : chaque enregistrement à la suite, toutes ses colonnes ensemble, comme dans un CSV. En colonnes : toutes les valeurs d'une colonne à la suite, puis celles de la colonne suivante.

|                                       | Stockage en lignes                                                                          | Stockage en colonnes                                      |
|---------------------------------------|---------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Ce qu'il écrit à la suite             | un enregistrement entier                                                                    | toutes les valeurs d'une colonne                          |
| Lire une colonne sur quatre           | lit les quatre                                                                              | lit celle-là                                              |
| Ajouter ou modifier un enregistrement | facile, au bout du fichier                                                                  | coûteux : on réécrit des blocs entiers                    |
| Compression                           | médiocre : des valeurs de natures différentes se côtoient                                   | forte : des valeurs semblables se suivent                 |
| Fait pour                             | les applications qui traitent un enregistrement à la fois : l'ERP, le portail des garanties | l'analyse, qui lit peu de colonnes sur beaucoup de lignes |

Une requête analytique lit en général peu de colonnes sur beaucoup de lignes. La vibration maximale du compresseur 2 sur trois mois ne demande que deux colonnes de l'historien, TagName et Value, sur 275 436 lignes. En lignes, il faut lire les quatre colonnes de toutes les lignes, horodatages et codes de qualité compris. En colonnes, on ne lit que les deux utiles, et elles se compressent bien : TagName ne prend que 58 valeurs différentes, qui se répètent sans cesse.

C'est la raison de fond du format colonnaire en analyse : lire moins d'octets, et des octets mieux compressés. Les bases transactionnelles, comme celle de l'ERP de Boréal, restent en lignes, parce qu'elles écrivent et relisent une commande à la fois.

## 2. Cinq formats, ce qu'ils coûtent et ce qu'ils rendent

Labo : Labo\_02.md, étapes 2 et 3.

| Format     | Organisation                      | Schéma et types                                           | Compression                           | Lisible à l'œil | Usage typique                                              |
|------------|-----------------------------------|-----------------------------------------------------------|---------------------------------------|-----------------|------------------------------------------------------------|
| CSV        | lignes, texte                     | aucun : tout est texte, l'en-tête donne au mieux des noms | faible, ou par gzip autour du fichier | oui             | l'export d'un système : l'historien, les extraits de l'ERP |
| JSON lines | lignes, texte, un objet par ligne | aucun de déclaré ; imbrication permise                    | faible, ou par gzip                   | oui             | les événements et les API : la télémétrie de Boréal        |
| Avro       | lignes, binaire                   | schéma embarqué, évolution prévue                         | par blocs                             | non             | les flux d'événements, écrits un à un                      |
| Parquet    | colonnes, binaire                 | schéma et types embarqués, imbrication permise            | forte, par colonne                    | non             | le lac analytique : le bronze et l'argent de ce parcours   |
| ORC        | colonnes, binaire                 | schéma et types embarqués                                 | forte, par colonne                    | non             | l'écosystème de Hadoop et de Hive                          |

Le CSV et le JSON restent partout parce qu'ils sont universels : tout système sait en écrire, tout humain sait en lire. Ce sont d'excellents formats d'échange et de mauvais formats de stockage. Avro et Parquet se partagent le reste du travail : Avro pour écrire vite, un enregistrement à la fois, dans un flux ; Parquet pour lire vite, une colonne à la fois, dans un lac. ORC fait le même travail que Parquet ; on le croise surtout là où Hive a régné, et ce parcours n'en a pas l'usage.

Chez Boréal, les sources arrivent en CSV et en JSON lines, parce que c'est ce que leurs systèmes exportent. Le lac, lui, est en Parquet : l'historien des 92 jours passe de 13,7 Mo en CSV à 1,5 Mo dans le bronze, la télémétrie de 70,6 Mo en JSON lines à 10,8 Mo.

### 3. Parquet, de près

Labo : Labo\_02.md, étape 2.

Un fichier Parquet se découpe en trois niveaux :

| Niveau                                  | Ce que c'est                                                   | Pourquoi il existe                                                |
|-----------------------------------------|----------------------------------------------------------------|-------------------------------------------------------------------|
| groupe de lignes ( <i>row group</i> )   | un paquet de lignes ; DuckDB en met jusqu'à 122 880 par groupe | l'unité qu'un moteur peut lire ou sauter, et traiter en parallèle |
| bloc de colonne ( <i>column chunk</i> ) | les valeurs d'une colonne dans un groupe de lignes             | la lecture colonne par colonne                                    |

| Niveau | Ce que c'est                                            | Pourquoi il existe                         |
|--------|---------------------------------------------------------|--------------------------------------------|
| page   | une tranche d'un bloc de colonne, encodée et compressée | l'unité de compression et de décompression |

À la fin du fichier, le pied de page (*footer*) décrit le tout : le schéma, avec le nom et le type de chaque colonne, et, pour chaque bloc de colonne, le nombre de valeurs, le minimum et le maximum. Un moteur lit ce pied de page avant le reste. C'est lui qui rend Parquet autonome : aucun type à deviner, aucune documentation à consulter pour savoir ce que contient une colonne.

Deux mécanismes réduisent la taille. L'encodage d'abord, qui tire parti de la forme des valeurs : un dictionnaire remplace les 58 étiquettes répétées de l'historien par de petits numéros, et un codage par plages (*run-length encoding*) écrit « 192, répété 4 000 fois » plutôt que 4 000 fois 192. La compression ensuite, appliquée à chaque page : snappy, rapide et modérée, le choix par défaut de beaucoup d'outils, DuckDB compris ; zstd, un peu plus lente, nettement plus compacte. Sur l'historien écrit en un seul fichier : 4,0 Mo sans compression, 1,8 Mo en snappy, 1,2 Mo en zstd. Le bronze de ce parcours prend zstd : il se lit bien plus souvent qu'il ne s'écrit.

Deux mécanismes réduisent la lecture. La projection : le moteur ne lit que les blocs des colonnes que la requête nomme. Le prédicat poussé (*predicate pushdown*) : avant de lire un groupe de lignes, le moteur compare le filtre de la requête aux statistiques du pied de page, et saute le groupe si aucune ligne ne peut y répondre. Un filtre `Value > 20` saute tout groupe dont le maximum est 12.

Piège — des statistiques qui ne servent à rien. Le minimum et le maximum n'aident que si les valeurs voisines sont regroupées. Dans un fichier quotidien de l'historien, `TagName` va de `ESS.BANC.REGIME` à `UTL.SEC.POINT_ROSEE` dans le même groupe de lignes : un filtre sur une étiquette ne saute rien. Trier les lignes par étiquette à l'écriture rendrait ces statistiques utiles. L'ordre des lignes dans un fichier est un choix, pas un hasard.

## 4. La règle du bronze

*Labo : Labo\_02.md, étapes 1 et 3.*

Le Cours 01 a montré ce que coûte un type deviné : une date SAP devenue entier, un code devenu nombre, un horodatage qui perd son décalage, et aucune erreur pour le signaler.

La règle du bronze en découle : on déclare les types, on ne les devine pas. Et l'on ne déclare un type que si la conversion ne perd rien.

| Ce que livre la source                                 | Dans le bronze             | Pourquoi                                                       |
|--------------------------------------------------------|----------------------------|----------------------------------------------------------------|
| un horodatage à format fixe, sans fuseau (l'historien) | TIMESTAMP                  | rien ne se perd : le fichier ne portait pas de fuseau non plus |
| un horodatage avec son décalage (la télémétrie)        | texte                      | le convertir jetterait le décalage                             |
| un nombre, vide quand la mesure manque                 | DOUBLE, le vide devenu nul | la valeur et son absence survivent                             |

| Ce que livre la source                           | Dans le bronze | Pourquoi                                                     |
|--------------------------------------------------|----------------|--------------------------------------------------------------|
| un code de qualité entier                        | INTEGER        | le dictionnaire le garantit                                  |
| un code, un numéro à zéros de tête, une date SAP | texte          | la conversion appartient à celui qui connaît la règle métier |

Le montant mérite une ligne à part. Un DOUBLE représente mal les centimes, et une somme de montants en DOUBLE peut tomber à un cent près plutôt que juste. Un montant se convertit en DECIMAL, jamais en DOUBLE. Chez Boréal, les montants de l'ERP n'entrent pas dans le bronze : ils vont dans la couche RAW, en texte, au Cours 03.

## 5. Le stockage objet et le partitionnement

Labo : Labo\_02.md, étapes 1, 2 et 3.

Un lac d'entreprise vit rarement sur un disque. Il vit dans un stockage objet : Amazon S3, Azure Blob Storage, Google Cloud Storage. Trois différences avec un système de fichiers changent la façon d'y ranger les données.

| Dans un stockage objet                                                                                                             | Ce que ça change                                                                                  |
|------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| pas de vrais dossiers, seulement des clés :<br>lac/bronze/usine/historien/jour=2025-10-28/data_0.parquet est un nom, pas un chemin | lister un « dossier » est une recherche par préfixe, qui coûte d'autant plus qu'il y a d'objets   |
| un objet ne se modifie pas : il se remplace en entier                                                                              | on n'ajoute pas une ligne à un fichier du lac ; on réécrit le fichier, ou l'on en ajoute un autre |
| on paie à la requête et à l'octet transféré                                                                                        | lire moins de fichiers et moins d'octets économise de l'argent, pas seulement du temps            |

Azure Data Lake Storage ajoute de vrais dossiers au stockage objet d'Azure, mais la conduite reste la même. Le lac de ce parcours vit sur votre disque, dans plateforme/lac/, et il se range comme s'il vivait dans un stockage objet : les mêmes clés, les mêmes règles.

Le partitionnement par chemin met une valeur dans la clé : jour=2025-10-28/. C'est la convention de Hive, que DuckDB, Spark et Databricks reconnaissent d'eux-mêmes : la partition devient une colonne, jour, et un filtre sur elle permet de ne pas même ouvrir les autres fichiers. C'est l'élagage des partitions (*partition pruning*), plus radical que le prédicat poussé : sur l'historien, un filtre sur le 28 octobre ne lit qu'un fichier sur 92.

Choisir la clé de partition est une décision d'ingénierie. Trois critères comptent.

| Critère               | Ce qu'il demande                                                   | Chez Boréal                                                                                                 |
|-----------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| Le grain de livraison | ce qu'on devra remplacer d'un bloc quand une livraison est reprise | un jour de l'historien, un jour de réception de la télémétrie                                               |
| Le filtre fréquent    | ce par quoi les consommateurs filtrent le plus souvent             | la date                                                                                                     |
| La cardinalité        | combien de partitions la clé produit                               | 92 jours : raisonnable ; une partition par étiquette et par jour en ferait 5 336, presque toutes minuscules |

La partition suit le fichier livré, pas son contenu. Le fichier de l'historien du 1er octobre commence par des valeurs de départ datées du 30 septembre : elles restent dans `jour=2025-10-01`, avec le reste de leur livraison. Ce choix prépare le Cours 03 : quand un jour est livré de nouveau, on remplace sa partition, et rien d'autre.

Piège — les petits fichiers. Chaque fichier a un coût fixe : l'ouvrir, lire son pied de page, et, dans un stockage objet, payer une requête de plus. Des milliers de petits fichiers coûtent plus cher à lire que quelques gros, à contenu égal. La télémétrie de Boréal, partitionnée par jour, fait 843 fichiers de 13 Ko en moyenne : les lire tous prend plus de dix fois plus de temps que lire le même contenu en un seul fichier. Le remède s'appelle le compactage (Cours 10) : réécrire périodiquement les petits fichiers en quelques gros, sans changer les données.

## 6. DuckDB sur le lac

*Labo : Labo\_02.md, étapes 1, 2 et 3.*

Le Cours 01 a présenté DuckDB, qui interroge un fichier sur place, sans serveur. Il lit et écrit aussi le Parquet sans rien installer de plus. Ce qui sert dans toute la suite de la formation :

| Geste                              | Instruction                                                                                                                     |
|------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| lire tout le bronze de l'historien | <code>FROM 'plateforme/lac/bronze/usine/historien/*/*.parquet' : la colonne jour apparaît d'elle-même</code>                    |
| déclarer les types d'un CSV        | <code>read_csv(..., columns = {'TagName': 'VARCHAR', ...}),</code> ou <code>all_varchar = true</code> pour tout garder en texte |
| déclarer la forme d'un JSON lines  | <code>read_json(..., columns = {...}),</code> structures imbriquées comprises                                                   |
| écrire du Parquet partitionné      | <code>COPY (...) TO 'dossier' (FORMAT parquet, COMPRESSION zstd, PARTITION_BY (jour), OVERWRITE)</code>                         |

| Geste                                            | Instruction                                      |
|--------------------------------------------------|--------------------------------------------------|
| lire le schéma embarqué                          | <code>parquet_schema('fichier.parquet')</code>   |
| lire les groupes de lignes et leurs statistiques | <code>parquet_metadata('fichier.parquet')</code> |
| voir combien de fichiers une requête lit         | <code>EXPLAIN ANALYZE SELECT ...</code>          |

Deux comportements de COPY se retiennent, parce qu'ils arrêtent le travail. DuckDB crée le dossier final, mais pas ses parents : il faut créer `plateforme/lac/bronze/usine/` avant d'écrire. Et il refuse d'écrire dans un dossier qui n'est pas vide, sauf avec `OVERWRITE`, qui le vide d'abord. Relancer un script de bronze redonne alors exactement le même bronze : c'est un premier pas vers l'idempotence du Cours 03.

Le format ne fait pas tout le gain de vitesse. Sur l'historien, une requête prend environ 3 secondes sur les CSV quand DuckDB devine les types, 0,6 seconde quand on les déclare, et moins de deux centièmes sur le bronze. Deviner coûte cher : la détection des types se refait à chaque requête. Déclarer les types corrige à la fois la justesse et une bonne part de la lenteur ; le Parquet fait le reste.

Snowflake. Snowflake fait le partitionnement tout seul, en interne : chaque table se découpe en micro-partitions de 50 à 500 Mo de données non compressées, et Snowflake garde pour chacune l'étendue des valeurs de chaque colonne. Une requête saute les micro-partitions que ses filtres excluent, exactement comme DuckDB saute un groupe de lignes Parquet. On ne choisit pas de chemin ; on peut seulement demander, par une clé de regroupement (*clustering key*), que les valeurs voisines se retrouvent ensemble.

Databricks. Une table Delta, le format de Databricks, est faite de fichiers Parquet et d'un journal de transactions. Le piège des petits fichiers y existe aussi, et Databricks le traite par `OPTIMIZE`, qui compacte ; l'optimisation prédictive le lance d'elle-même sur les tables gérées par Unity Catalog. Databricks recommande désormais le regroupement dynamique (*liquid clustering*) plutôt que le partitionnement par chemin. Le Cours 10 y revient.

Recoupements — ce que ce cours partage avec ses voisines. - Snowflake. Les micro-partitions de Snowflake sont le partitionnement d'un lac, appliqué en interne et sans chemin à choisir. La formation Snowflake possède la plateforme ; ici, on ne regarde que le principe commun : lire moins, en sautant ce que les statistiques excluent.

---

## Points clés à retenir

- Un format en colonnes ne lit que les colonnes demandées et saute ce que ses statistiques excluent : c'est le format de l'analyse. Les applications transactionnelles restent en lignes.
- CSV et JSON sont des formats d'échange ; Parquet est le format du lac ; Avro sert les flux.
- Parquet porte son schéma et ses statistiques, et se compresse colonne par colonne : l'historien passe de 13,7 Mo à 1,5 Mo.
- Le bronze déclare ses types, et garde en texte ce qu'une conversion abîmerait : les décalages horaires, les codes, les dates SAP, les montants.

- Dans un lac, le chemin sert d'index : on partitionne par jour livré, on élague au filtre, et l'on se méfie des petits fichiers.
  - COPY ne crée pas les dossiers parents, et refuse un dossier plein sans OVERWRITE.
- 

## Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_02.md`. Faites-les après le labo : plusieurs énoncés supposent que le bronze existe.